

A Small Convolutional Key Classifier Trained on Beatport Annotations

MIREX 2026 Audio Key Detection: system description and dataset-usage disclosure

John Hurliman

Independent researcher, Oakland, CA, USA

jhurliman@jhurliman.org <https://github.com/jhurliman/jams>

Abstract

We submit K10, a 109k-parameter convolutional network that predicts one of 24 global keys from a log-magnitude constant-Q transform of the whole track. It is trained only on the `beatport_key` annotations distributed through mirdata (the GiantSteps-MTG-Keys corpus with revised labels), selected by cross-validation inside that corpus, and run as a single forward pass on CPU. The submission is inference-only, MIT-licensed, and packaged as one script with bundled weights. This abstract describes the system, states precisely how GiantSteps Key was used during the project’s development, and reports the development-time GiantSteps Key scores under both fifth-credit conventions so that MIREX’s results can be read against them.

1 System

Input features. The audio is decoded, downmixed to mono, and resampled to 22,050 Hz. A constant-Q transform with 24 bins per octave is computed over 8 octaves from C1 with hop 4,096 (about 5.4 frames per second), with a 4-semitone margin on each side that is cropped away after the transform (208 bins computed, the central 192 kept). Magnitudes are compressed with $\log(1 + x)$. The full track is one input; nothing is cropped or windowed at inference time.

Architecture. Three convolutional blocks, each two 3×3 convolutions with batch normalization and ELU followed by 2×2 max pooling (16, 32, and 64 channels), then dropout, then average pooling over *time only*. Pooling over frequency is deliberately avoided: it would make the network invariant to transposition, and key identity lives in absolute frequency position. The pooled 64×24 map feeds one linear layer with a 24-way softmax over $\{12 \text{ tonics}\} \times \{\text{major, minor}\}$. The model has 109k parameters (0.45 MB). The design follows the convolutional key-classifier family of Korzeniewski and Widmer [1]; it is smaller and has no ensemble.

Inference. One forward pass over the whole-track CQT, argmax over the 24 classes, no test-time augmentation. The output is written as a single tab-separated line, tonic (sharps only) and mode, e.g. `C_major` with a tab separator. The wrapper sets `torch` to one thread; a two-minute track takes about 1.7 s wall on one Apple-Silicon CPU core including interpreter start and model load, with about 390 MB peak memory. No GPU and no network access are used at pre-

diction time.

Training. The corpus is `mirdata beatport_key` [2]: 1,486 Beatport preview excerpts with revised key annotations, of which 1,363 remain after dropping unparseable or non-major/minor labels and undecodable audio. Augmentation is a pitch shift of ± 4 semitones applied as a roll of the CQT bins with the label transposed accordingly. Optimization is AdamW with a cosine schedule on about 60-second random crops. Architecture width and the epoch budget were chosen by 5-fold cross-validation within the training corpus; the submitted weights are the model trained on the full corpus at the selected budget. The whole procedure is committed as `eval/train_key_cnn.py` in the repository, and the run cost about two hours on one A10 GPU.

2 Dataset usage and development history

The task rules ask for an accurate account of dataset use. The account below is taken from the project’s public experiment ledger, which records every evaluation with its date and commit.

Training data. K10 was trained only on `beatport_key`. That corpus contains all 1,157 usable GiantSteps-MTG-Keys training identifiers (it is the same Beatport collection with revised annotations, not an independent set) and has zero track-identifier overlap with GiantSteps Key [3]; the check is committed as `eval/verify_key_disjoint.py` and compares Beatport identifiers from the two mirdata indexes (1,486 versus 600, empty intersection). No Isophonics or other key data was used. The check does not rule out duplicate recordings under different identifiers.

Prior use of GiantSteps Key in this project. Before K10 existed, the project evaluated several key systems on the usable 567 of the 604 GiantSteps Key excerpts (atonal, multi-key, and unparseable labels dropped; 600 had retrievable audio): a template matcher (Essentia’s `edma` profile [4, 5]), the self-supervised S-KEY model [6], madmom’s CNN key processor [1, 7], and two small logistic-regression refinements of the template output that were fit on GiantSteps-MTG-Keys. One earlier mode classifier had been fit on GiantSteps Key itself; it was identified as contaminated, retired, and replaced by the GiantSteps-MTG-fit version before any comparison was reported. The decision

to build K10 came from analysing the banked GiantSteps Key predictions of those systems: on tracks where madmom was right and our fusion system was wrong, most errors were unreachable by any reranking of the existing candidates, which argued for a stronger base model rather than a better combiner. In that sense the benchmark informed the *decision to train a CNN*, though not its architecture, hyperparameters, or weights. Two implementation bugs were caught by chance-level cross-validation scores on the training corpus and fixed before any test-set involvement.

Test use of GiantSteps Key by K10. The experiment was pre-registered in the ledger before training, with a fixed decision rule. All selection (width, epoch budget, and the choice between the standalone CNN and two fusion variants that reused the existing combiner) used training-corpus cross-validation only. The locked model was then evaluated exactly once on the 567-track subset. A post-hoc per-genre breakdown of that single evaluation was computed afterwards and is descriptive only. No further training or tuning followed. The weights in this submission are byte-identical to that evaluated checkpoint (MD5 6141daf25376c16a7bc4326b742e4a3c).

3 Development-time results on GiantSteps Key

Table 1 reports the single evaluation, scored two ways. The project’s scorer awards 0.5 for a same-mode key a perfect fifth away in *either* direction, as in the 2018 definition [1]; `mir_eval` 0.8.2 [8] awards it only for the ascending fifth. The task organizers have stated that both conventions will be reported, so both are given. Exact, relative (0.3), and parallel (0.2) credits are identical under the two conventions. Confidence intervals are 95% percentile bootstraps over tracks (10,000 resamples, seed 0).

These numbers are development-time results on a subset the project had already used, so they should be read as an expectation, not as a prediction of the MIREX score: MIREX evaluates the full 604-excerpt set with its own scorer, and a subset mean is not comparable to a full-set mean. In particular, the project earlier attributed a gap between madmom’s published full-set score (0.746) and its subset score here (0.833) to subset selection; that explanation was withdrawn because it is arithmetically impossible under its own assumptions ($604 \times 0.746 / 567 = 0.795 < 0.833$), and the published figure describes a single selected model whereas the default processor is an ensemble. Descriptively, K10’s weighted score by genre on the subset ranges from 0.900 (Dubstep) to 0.739 (Electronica/Downtempo), with Trance (0.763) also below the mean; sustained, modally ambiguous material remains the main failure mode.

4 Submission contents and reproducibility

The package contains `predict_key.py` (the calling contract is `predict_key.py %input %output`;

PEP 723 inline metadata pins `torch==2.8.*`, `librosa>=0.10`, `numpy>=1.26,<2.3`, Python 3.10–3.12, so `uv run predict_key.py in.wav out.txt` builds its own environment), the weights `key_cnn_v1.pt`, a README with the calling format, runtime, memory, and environment details, and this abstract. Inference reproduces the training featurization exactly. The training script, the disjointness check, the experiment ledger with the pre-registration and the single-shot result, and the per-track development predictions used for Table 1 are in the public repository. Code and weights are MIT-licensed; the madmom weights used only for the comparison above are CC BY-NC-SA and are not part of the submission.

Acknowledgments and AI use

Thanks to the MIREX organizers for clarifying the eligibility question in advance. Claude (Anthropic) assisted with implementation, experiment execution, and drafting under the author’s direction; OpenAI Codex reviewed the accompanying paper’s code and claims. The author is responsible for the submission.

References

- [1] F. Korzeniowski and G. Widmer, “Genre-agnostic key classification with convolutional neural networks,” in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018.
- [2] Á. Faraldo, “Tonality estimation in electronic dance music: A computational and musically informed examination,” Ph.D. dissertation, Universitat Pompeu Fabra, 2017, the GiantSteps-MTG-Keys dataset accompanies this work.
- [3] P. Knees, Á. Faraldo, P. Herrera, R. Vogl, S. Böck, F. Hörschläger, and M. Le Goff, “Two data sets for tempo estimation and key detection in electronic dance music annotated from user corrections,” in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR)*, Málaga, Spain, 2015.
- [4] Á. Faraldo, E. Gómez, S. Jordà, and P. Herrera, “Key estimation in electronic dance music,” in *Proceedings of the 38th European Conference on Information Retrieval (ECIR)*, Padua, Italy, 2016.
- [5] D. Bogdanov, N. Wack, E. Gómez, S. Gulati, P. Herrera, O. Mayor, G. Roma, J. Salamon, J. R. Zapata, and X. Serra, “Essentia: An audio analysis library for music information retrieval,” in *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*, Curitiba, Brazil, 2013.
- [6] Y. Kong, G. Meseguer-Brocal, V. Lostanlen, M. Lagrange, and R. Hennequin, “S-KEY: Self-supervised learning of major and minor keys from audio,” in *Proceedings of the IEEE International Conference on*

system	weighted, sym. fifth	weighted, asc. fifth	exact
K10 (this submission)	0.8321 [0.804, 0.859]	0.8145	0.7795
madmom CNN (default processor)	0.8328 [0.806, 0.858]	0.8134	0.7725
paired K10 – madmom	−0.0007 [−0.019, +0.018]	+0.0011 [−0.020, +0.022]	
wins / losses / ties	41 / 40 / 486	38 / 39 / 490	

Table 1: GiantSteps Key, 567 usable excerpts, one evaluation of the locked K10 model. madmom was run with its published weights and default settings on the same tracks. The paired difference crosses zero under both fifth conventions; no superiority or equivalence claim is made.

Acoustics, Speech and Signal Processing (ICASSP),
2025.

- [7] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and G. Widmer, “madmom: A new python audio and music signal processing library,” in *Proceedings of the 24th ACM International Conference on Multimedia*, Amsterdam, The Netherlands, 2016.
- [8] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, “mir_eval: A transparent implementation of common MIR metrics,” in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, Taipei, Taiwan, 2014.